

Document Packet Intelligence & Evidence Retrieval

Project journey — prepared for Parakit AI

How this system was built, in the order it actually happened: what we found, what we changed, what we tried that failed, and what the numbers honestly say. Every figure quoted here is produced by the scripts in `scripts/` and written to `outputs/benchmarks/`.

1. The brief, and how we read it

Three stages over a PDF packet holding several unrelated documents: detect boundaries and classify each document, convert each into a structured representation, and retrieve evidence for a query. The brief is explicit that *"the strongest submissions are not necessarily those with the highest benchmark scores"* and asks for engineering judgement, experimentation methodology and honest evaluation.

We took that at face value. The working rule for the whole project became: **a number we cannot defend is worth less than a smaller number we can**. That rule ended up reversing several decisions, including which model we shipped.

We framed the problem as three questions that must be answered in order:

1. Where does one document end and the next begin? — a decision over the $N-1$ adjacent page pairs, not over pages, which makes grouping a deterministic consequence rather than a second model that can disagree with the first.
2. What is each document, and how sure are we? — type prediction needs an abstention path.
3. What is inside it, and where exactly? — retrieval must cite a document and a page, so structure and page provenance are the substrate the answer is built from.

Three stages, with dataclass contracts between them: `PageRepresentation` → `DocumentGroup` → `Chunk` → `EvidenceResult`. The contracts turned out to be the most valuable design decision — they let us swap the Stage 3 encoder three times without touching Stage 1 or Stage 2.

2. Dataset decision

The brief named DocSplit v2. It has no public train/validation split, and your clarification confirmed OpenPSS (`nutrientdocs/openpss-mirror`) as the development dataset. We used OpenPSS SHORT for boundary detection.

OpenPSS carries **no document-type labels**, so it cannot train a type classifier. We added **RVL-CDIP OCR text** (`albertklorer/rvl_cdip_ocr`) — the standard 16-class document taxonomy, shipped with OCR words already extracted, which mattered because the target environment has no OCR binary installed.

One environment constraint shaped the whole data layer: this machine runs Python 3.14, where `pyarrow` has no wheel and fails to build without a CMake toolchain — so `datasets` was unusable. We fetch everything through the HuggingFace datasets-server REST API instead. The result is that the pipeline reproduces with `pip install -r requirements.txt` and nothing else.

Late in the project we discovered `nutrientdocs/doc-split-benchmark` is publicly readable (test split only). We scored against it but never trained on it. That single decision produced the most important finding in the project — §7.

3. Starting audit: a recurring pattern

Before writing anything we ran the existing pipeline on a real packet and inspected the output rather than reading the code. A nine-page packet containing four documents came back as **one document spanning all nine pages**.

The cause was not a tuning problem. `config/default.yaml` declared `BAAI/bge-small-en-v1.5` and a CLIP visual model; neither was wired into feature extraction anywhere. The "semantic" text feature was a hashed bag-of-words explicitly commented as *not* a pretrained embedding, and the "visual" feature was a 16-bin grey histogram. The two features with the most power to separate different document types were running on crude stand-ins.

This pattern — **capability declared in config, never wired in code** — recurred in every stage, and finding it became the standing first move:

Stage	Declared	Reality
1	<code>tfidf_logistic_regression</code> classifier	<code>model_path: null</code> → silent keyword fallback
2	table extraction module	never imported by the parser; zero tables ever extracted
2	LLM fallback contract	never called by anything
3	dense semantic index	hashed bag-of-words; effectively lexical

We adopted a rule in response: **verify against real output, not source code**. Every defect below was found by inspecting what the system actually produced.

4. Stage 1 — boundaries and document types

Boundary detection

We trained a calibrated HistGradientBoosting classifier over 21 pairwise page features on OpenPSS (95 streams, 15,905 page pairs). Isotonic calibration was added because raw GBM probabilities on an imbalanced split compress toward zero, making any fixed threshold unreliable.

One deliberately non-learned addition mattered more than the model: **a printed page number resetting is near-conclusive evidence of a boundary, and unlike the learned score it is domain-invariant**. Letting it override a muted learned probability is what took the verification packet from one merged blob to four correct documents.

Document classification

The config declared a trained classifier but `model_path` was `null`, so the system silently fell back to keyword matching — a misconfigured run was indistinguishable from a trained one. Worse, the confidence was "fraction of this class's keyword list matched", which cannot express ambiguity: text reading only `"Total amount 500"` returned **invoice @ 0.2**.

We trained a real classifier on 11,109 RVL-CDIP documents: **0.807 accuracy, 0.786 macro-F1**, with the two most relevant classes scoring highest (resume F1 0.952, invoice F1 0.885).

Two design decisions came out of this:

- **Explicit abstention.** Confidence is now a normalised distribution gated by absolute evidence, and `min_confidence` is enforced. `"Total amount 500"` now returns **unknown @ 0.111**.
- **A hybrid tier for classes with no training data.** RVL-CDIP has no `passport` or `bank_statement` class and no public labelled corpus exists, so a weighted lexicon covers them. Critically, the trained model's confidence is treated as **not comparable** for an out-of-taxonomy type — for a passport the model must assign probability mass to some unrelated class, so it is confidently wrong by construction. A regression test caught exactly this: a stub returning `letter @ 0.90` beat a correct `passport @ 0.82` under naive numeric comparison.

A feature-size sweep produced a result worth noting: **20k TF-IDF features beat 200k** (0.807 vs 0.803 accuracy) at **one tenth the size** — 3.3 MB against 33.3 MB. The larger space was mostly noisy OCR tokens. We kept the smaller model.

5. Stage 2 — document structuring

Inspecting the structured output surfaced five defects, all invisible from the code:

1. **Tables were never extracted.** The extractor existed but nothing imported it. Later, when wired in, the detection *gate* rejected a generic report table because its keyword list was invoice/bank vocabulary. `pdfplumber` was never even invoked.
2. **Lists were never detected,** because the check looked at the start of a whole block while extractors emit each item as its own block.
3. **Running headers corrupted the structure** — the letterhead became a heading on every page, and page-number blocks merged into real headings, producing titles like `"Page 2Service Details"`.
4. **Field extraction was wrong:** `total` matched inside "Subtotal" and reported 84,500 when the real total was 99,710; `passport_number` captured the word "Information" from "Passport Information".
5. **A span-joining bug in the PDF parser** concatenated text with no separator, producing `MetricBaselineTargetUnit`, and joined lines with no newline — which silently broke list detection, since that splits on newlines.

Defect 5 is the one worth dwelling on. An external review of the output diagnosed the table problem as "extraction exists but is not propagating into the structure/chunker/API". That diagnosis was wrong: propagation worked fine. The failures were all at *detection*, upstream. Following the diagnosis literally would have meant rewriting correct code while leaving the real bugs in place.

We also found the engine produced **zero structure on 2,500 real OpenPSS pages** — because every structural signal derives from bounding boxes and font sizes, which OCR-only sources lack. A text-only path reconstructing structure from typographic convention fixed it:

Signal	Before	After
Sections	12	2,575
Elements	0	31,039
Mean chunk tokens	256.6 (fixed window)	141.3 (semantic)
Text retention	—	97.7%

On annotated fixtures the structure engine scores **1.00 F1 on headings, tables, lists and captions**. We state plainly in the report that those fixtures are authored, so they measure conformance to constructs we chose rather than generalisation — which is exactly why the label-free coverage run above sits beside them.

6. Stage 3 — evidence retrieval

The dense index used a hashed bag-of-words whose slots are `int.from_bytes(token) % dims`, so related words land in unrelated slots. Measured in isolation it scored **R@1 = 0.086**.

The consequence was worse than a weak component. Fusing it with BM25 produced **R@1 = 0.257 against BM25-alone's 0.714** — the shipped default configuration was substantially *worse than deleting half of it*. No aggregate score would have revealed that; it took an ablation.

We made the encoder pluggable and measured all three:

Configuration	R@1	MRR	Index time
bm25_only	0.714	0.774	1.0 s
rfr_hashed (old default)	0.257	0.407	1.1 s
rfr_svd + reranker (shipped)	0.771	0.821	7.5 s
rfr_bge + reranker	0.886	0.929	67 s

We shipped the lower-scoring option. bge wins by +0.115 R@1 but requires ~1 GB of torch, which took two install attempts and a DLL failure to get working on this machine. A submission that does not install is worth less than 0.115 R@1. The transformer is one config line away, with its numbers documented in both the config and `requirements.txt`.

We also replaced a reranker that *raised an exception* whenever its documented flag was enabled with a real deterministic one (term coverage, phrase runs, identifier exactness). It adds **+0.171 R@1 for roughly zero milliseconds and no dependency** — the best value in the table.

7. The measurement correction

This phase changed the project more than any model change.

Our own headline metric was measured wrong. The evaluation script selected the F1-optimal threshold *on the same data it then scored*, and the test set held only 12 streams. Splitting that set in half gave F1 **0.21** on one half and **0.44** on the other — for the same model.

We enlarged the test set to the full 108-stream split and separated threshold selection from scoring (splitting by stream, since pairs within a stream are not independent):

Test set	Honest held-out F1	Threshold-on-test F1	Optimism gap
12 streams	0.210	0.377	0.167
108 streams	0.379	0.406	0.027

Then we tried four times to raise it, and **all four failed**:

Attempt	Live features	F1
Baseline	6 / 14	0.379
Pseudo-layout reconstructed from OCR text	10 / 14	0.359
Natural class prior instead of balanced weighting	6 / 14	0.371
Seven sentence-flow features	13 / 21	0.371
Synthetic RVL-CDIP packets with real geometry	18 / 21	0.517

The *shape* of the failures was the finding. The first three traded precision up against recall down and landed on the same F1 — they moved **along one precision/recall curve** rather than shifting it. The threshold already exposes that trade-off, so recalibration cannot help; the ceiling is representational.

The fourth looked like a breakthrough — F1 0.517 against 0.379 — and was not. **F1 cannot be compared across corpora with different boundary densities.** At that corpus's 32% density, a classifier that marks every pair a boundary already scores 0.486. Lift over that trivial baseline was +0.031, against the baseline's +0.191. The higher-scoring model was substantially worse.

We added `lift_over_trivial` to the metric set at that point. It immediately paid for itself.

The DocSplit result, and the bug it exposed

Scoring against the actual benchmark produced the sharpest lesson of the project — in two stages, because our first reading of it was wrong.

First reading. Boundary F1 came out at 0.823, which looked like the best number in the project. It was not: at DocSplit's 72% boundary density, a classifier that marks *every* pair a boundary already scores 0.815. Lift over that trivial baseline was **+0.008**. Reported as raw F1 this would have been our headline result; measured as lift it was nothing.

Second reading — the one that mattered. Asked whether the system would actually perform if Parakit evaluated it, we measured what an evaluator would *see* rather than what our benchmark script reported. The 0.823 had been obtained at a

threshold refitted on DocSplit. At the **shipped** threshold the same model scored:

Decision rule	Boundary F1	Page grouping accuracy	Documents found vs actual
Fixed threshold 0.633 (shipped at the time)	0.110	0.216	1.2 vs 3.5
Expected-count (shipped now)	0.537	0.615	2.2 vs 3.5
Threshold refitted on DocSplit itself	0.851	0.868	4.2 vs 3.5

The shipped configuration recovered **6% of boundaries** and merged 3.5 documents into 1.2. The threshold had been fitted on OpenPSS, where 11% of adjacent pairs are boundaries; DocSplit packets are short and 72% of pairs are. One global threshold cannot serve both regimes, and refitting it on the target corpus would be benchmark-fitting.

The fix used something already in the system. The model is isotonic-calibrated, so **for calibrated probabilities their sum over a packet estimates the expected number of boundaries in it**. Splitting at the highest-scoring K pairs, $K = \text{round}(\text{sum}(p))$, adapts per packet from the model's own output — no threshold, no target labels. Page grouping accuracy on the target regime rose **0.216** → **0.615**, against a cost of 0.012 on the training regime.

We tried a length-based adaptive threshold first and rejected it on evidence: OpenPSS's 2–6 page streams average 22.8% boundary density against 17.1% for its longest — nowhere near 72%. The training corpus carries no signal that would let a length prior anticipate the target regime, which is exactly why the calibration-based rule, needing no such prior, was the right choice.

The underlying cause is a **regime inversion**:

Corpus	Median pages/stream	Boundary rate	Rare, informative class
OpenPSS (trained on)	21	11.4%	boundary
DocSplit (target)	4	72.3%	continuation

The model learned that boundaries need strong evidence, because in its training corpus they are rare. In the target corpus the rare and informative event is a *continuation*. The decision rule mitigates this; only retraining in the target regime removes it.

Was the learner ever the problem?

Late on, we were asked why the boundary model used scikit-learn's HistGradientBoosting rather than XGBoost, LightGBM or CatBoost. The honest answer was that it had been **inherited, not chosen** — and that `xgboost` had been sitting in `requirements.txt` imported by nothing, the same declared-but-unused pattern we had been correcting everywhere else.

So we measured all four on identical features, calibration, packet split and decision rule:

Learner	Boundary F1	Lift	Page grouping	Train time	Model size
scikit-learn HistGradientBoosting (shipped)	0.582	+0.163	0.825	9.6 s	0.8 MB
XGBoost	0.581	+0.163	0.829	1.5 s	0.8 MB
LightGBM	0.586	+0.168	0.828	0.7 s	0.8 MB
CatBoost	0.586	+0.168	0.806	2.9 s	0.5 MB

All four land within **0.005 F1 and 0.005 lift**. This was the third independent confirmation that the Stage 1 ceiling is representational: changing features did not move it, changing calibration did not move it, and changing the model family does not move it either.

scikit-learn was retained — the differences are inside noise, it is already a dependency, and `HistGradientBoostingClassifier` is itself a histogram GBM modelled on LightGBM. The one real difference is that LightGBM fits 13× faster, which matters for frequent retraining and not for a one-off. `xgboost` was removed from the dependency list, and all three alternatives are recorded there commented with their measured numbers.

8. Where the system honestly stands

Stage	Measurement	Result
1 — boundaries	OpenPSS, 108 held-out streams	F1 0.379, grouping accuracy 0.772
1 — boundaries	DocSplit <code>our200</code> (target task)	F1 0.537, grouping accuracy 0.615
1 — classification	RVL-CDIP held-out, 16 classes	0.807 acc / 0.786 macro-F1
2 — structure	annotated fixtures	heading/table/list/caption F1 1.00
2 — coverage	2,500 real OpenPSS pages	97.7% text retention, 1,250 pages/s
3 — retrieval	35 queries, 515 chunks	R@1 0.771, MRR 0.821
Resources	full pipeline	5.8 MB models , 153 MB RSS, CPU-only

Read **page grouping accuracy, not boundary F1**, on any corpus whose boundary density differs from training: at DocSplit's 72% density a classifier marking every pair a boundary scores F1 0.815, so F1 there is dominated by the base rate. Grouping accuracy

separates the shipped configuration from the one it replaced by 0.615 against 0.216, and is the honest signal.

Stated plainly: **the system segments long page streams well and short packets partially**. Both numbers appear because reporting only the first would overstate it and only the second would understate what the decision rule recovered.

Three areas carry no measurement, and we say so rather than filling them in:

- `passport` and `bank_statement` are lexicon-scored with no held-out evaluation — no public labelled corpus for them exists.
- Stage 2's 1.00 scores are on fixtures we authored.
- The retrieval evaluation is 35 queries, giving roughly ± 0.07 confidence intervals.

9. What we would do next

1. **Retrain in the target regime**. The DocSplit result is a training-distribution problem, not a model-quality one. `jordyv1/rvl_cdip_n_mp` — the multi-page RVL-CDIP variant DocSplit itself is assembled from — is public and provides genuine continuations rather than fabricated ones. This supersedes everything else.
2. **Label the extension classes**, closing the largest unmeasured area.
3. **Sequence modelling**. Every feature compares page i with $i+1$ in isolation, while this is a sequence-labelling problem with document-length priors. Untried, and the standard remedy for the representational ceiling we hit.
4. **Learned page-pair representations**. `text_delta` is TF-IDF cosine, which fires on shared vocabulary; a cross-encoder would model *continuation* rather than *overlap*.

10. The principles that shaped this

- **Verify against real output, not source code**. Every defect in this document was found by inspecting what the system produced. Four capabilities were declared in config and absent in code.
- **Ablate components; aggregate scores hide broken parts**. A working ranker and a broken one fused together scored worse than the working one alone, and only an ablation showed it.
- **Prefer a defensible number to a flattering one**. We corrected our own headline metric downward in method, and shipped a retrieval encoder that scores 0.115 lower because it reproduces anywhere.
- **Report what a metric cannot tell you**. Raw F1 is not comparable across corpora with different class balance. Adding lift-over-trivial reversed the apparent ranking of two models and exposed that our best-looking score was the do-nothing baseline.
- **Interrogate inherited choices**. The boundary learner and three dependencies arrived with the codebase and went unexamined until someone asked. Measuring settled it in minutes and removed a dead dependency; assuming would have left an unjustifiable answer in an interview.
- **A negative result is a result**. Six experiments failed here. Each one narrowed where the real problem lives, which is why the future-work list is specific rather than speculative.

This document accompanies `technical_report.pdf` (full methodology and benchmarks) and `docs/architecture.pdf` (system architecture). Section 9 of the technical report contains the AI usage declaration required by the brief; this journey document should be read alongside it.